

深圳大学实验报告

课程名称： 计算机系统 1

实验项目名称： 实验一

学院： 计算机科学与技术学院

专业： 计算机科学与技术

指导教师： 李志

报告人： 汉雨 学号： 2024040042 班级： .

实验时间： 20260424

实验报告提交时间： 20260424

教务处制

一、实验目的

实验目的：

- (1) 掌握处理器仿真工具 LC-3 软件的安装和使用方法。
- (2) 学会在 LC-3 仿真环境下编辑程序和转换成可执行目标程序的方法。
- (2) 学会在 LC-3 仿真环境下运行和调试程序的方法。

二、实验内容

实验要求：

利用提供的安装软件包和软件使用说明文档，完成以下实验内容：

- (1) 安装 LC-3 仿真器
- (2) 利用 LC3EDIT 输入机器代码程序（0/1 模式）并创建创建可执行目标程序。
- (3) 利用 LC3EDIT 输入机器代码程序（hex 模式）并创建创建可执行目标程序。
- (4) 利用 LC3EDIT 输入汇编代码程序并创建创建可执行目标程序。
- (5) 利用仿真器运用对应目标程序。
- (6) 学习和掌握断点，单步执行等调试方法和手段。

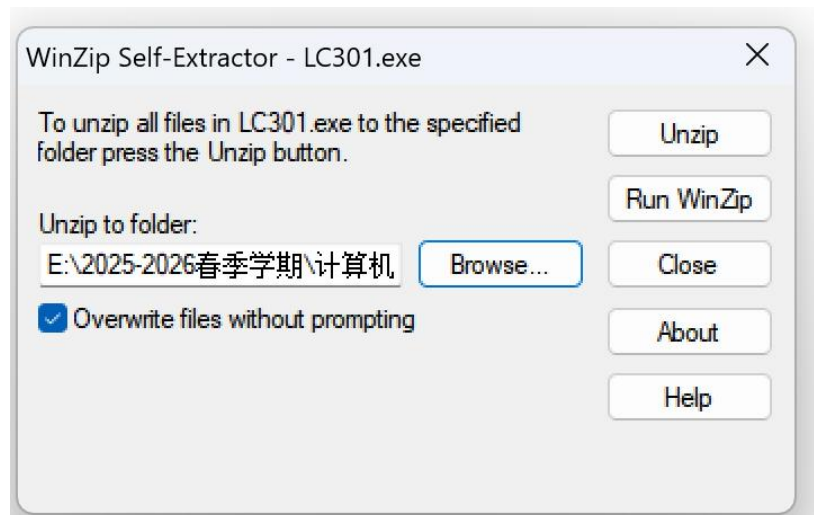
实验内容：

在 LC3winguide 中，通过 chapter1-3 学习和掌握仿真器的使用。完成 chapter4: P15 example1 和 P20 example2。(中文版本 P12example1 和 P17example2)

三、实验步骤与结果

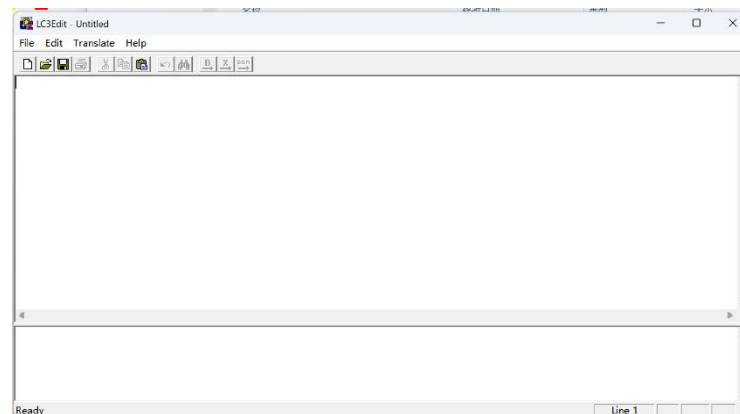
安装 LC-3 仿真器

运行“仿真器下的 LC301.exe”，选择对应的文件夹进行解压

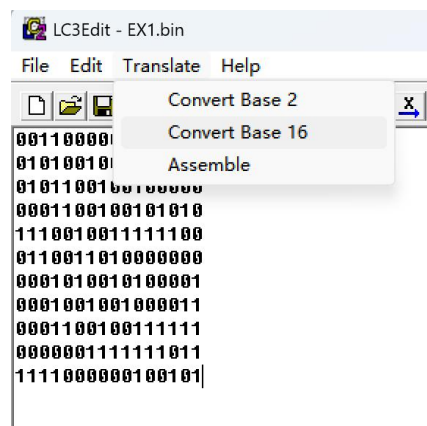


利用 LC3EDIT 输入机器代码程序（0/1 模式）并创建创建可执行目标程序。

下图为 LC3-Edit 界面



输入二进制代码后，保存为“EX1.bin”文件。选择“Translate-Convert Base2”进行编译。

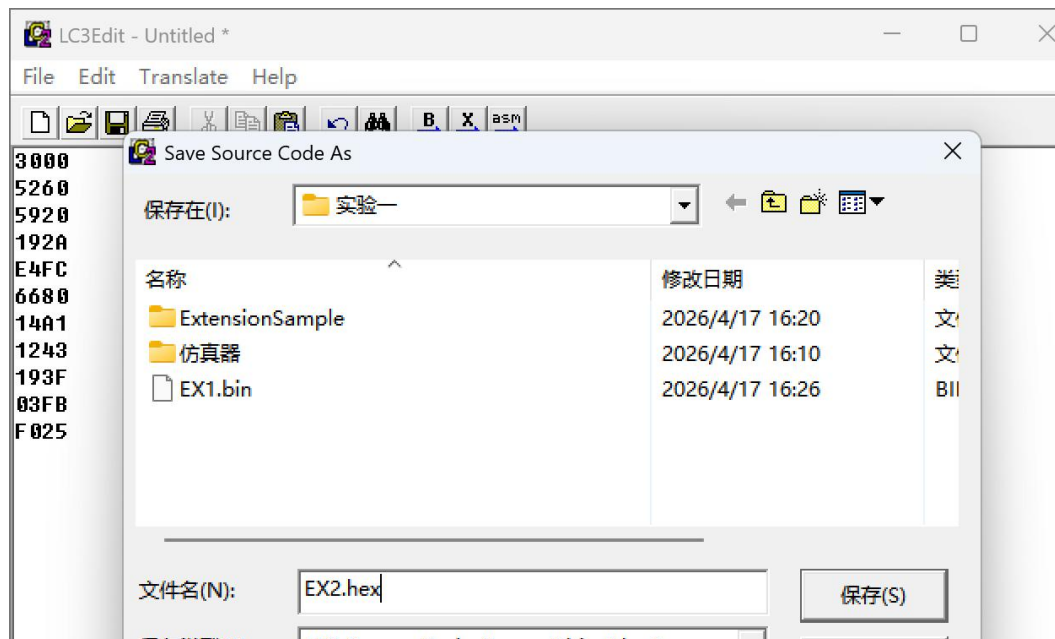


随后可见底部信息栏显示“0 error”

```
Converting E:\2025-2026春季学期\计算机系统一\实验一\EX1.bin from base 2...
Convert complete - 0 error(s)
```

利用 LC3EDIT 输入机器代码程序（hex 模式）并创建创建可执行目标程序。

输入所需实验的代码，选择保存为特定文件夹。



随后选择“Translate-Convert Base 16”进行编译。会出现下图“0 error”的信息，表示编译成功。

```
Converting E:\2025-2026春季学期\计算机系统一\实验一\EX2.hex from base 16...
Convert complete - 0 error(s)
```

Example 1

输入程序代码

```

0011 0010 0000 0000 ;程序起始地址: x3200
0101 010 010 1 00000 ; R2 复位
0001 010 010 0 00 100 ;R4中值与R2相加 结果放置与R2中
0001 101 101 1 11111 ;R5中值减去1
0000 011 11111101 ;如果结果>=0 转移至x3201
1111 0000 00100101 ;停止

```

模拟器加载 multiply.obj 文件

LC3 Simulator

File Execute Simulate Help

Jump to: x3000

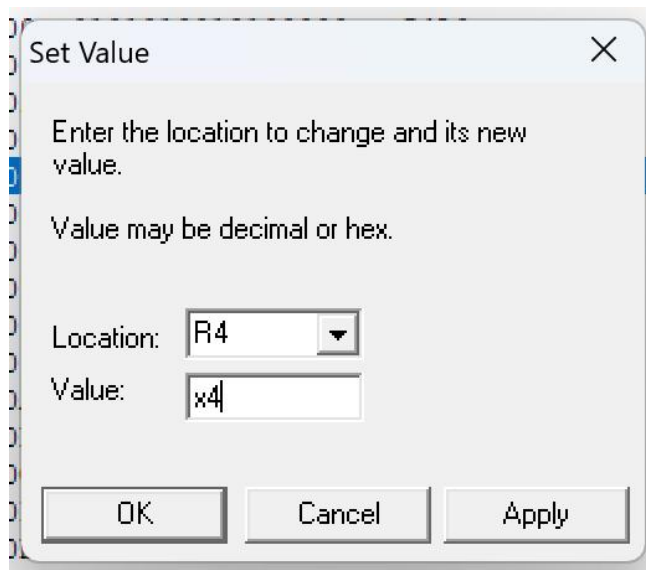
R0	x0000	0	R4	x0000	0	PC	x3000	12288
R1	x0000	0	R5	x0000	0	IR	x0000	0
R2	x0000	0	R6	x0000	0	PSR	x8002	-32766
R3	x0000	0	R7	x0000	0	CC	Z	

Load LC3 Object File

查找范围(I): 实验一

名称	修改日期	类型
ExtensionSample	2026/4/17 16:20	文件夹
仿真器	2026/4/17 16:10	文件夹
multiply.obj	2026/4/24 13:09	对象文件

设置 R4、R5 值



从 x3200 开始调试

R0	x0000	0	R4	x0005	5	PC	x3200	12800
R1	x0000	0	R5	x0003	3	IR	x0000	0
R2	x0000	0	R6	x0000	0	PSR	x8002	-32766
R3	x0000	0	R7	x0000	0	CC	Z	
▶ x3200	0101010010100000	x54A0				AND	R2, R2, #0	
▪ x3201	0001010010000100	x1484				ADD	R2, R2, R4	
▪ x3202	0001101101111111	x1B7F				ADD	R5, R5, #-1	
▪ x3203	0000011111111101	x07FD				BRZP	x3201	

以下为调试过程:

IR 指令正确, R2 为 0

R0	x0000	0	R4	x0005	5	PC	x3201	12801
R1	x0000	0	R5	x0003	3	IR	x54A0	21664
R2	x0000	0	R6	x0000	0	PSR	x8002	-32766
R3	x0000	0	R7	x0000	0	CC	Z	
▪ x3200	0101010010100000	x54A0				AND	R2, R2, #0	
▶ x3201	0001010010000100	x1484				ADD	R2, R2, R4	
▪ x3202	0001101101111111	x1B7F				ADD	R5, R5, #-1	
▪ x3203	0000011111111101	x07FD				BRZP	x3201	
▪ x3204	1111000000100101	xF025				TRAP	HALT	

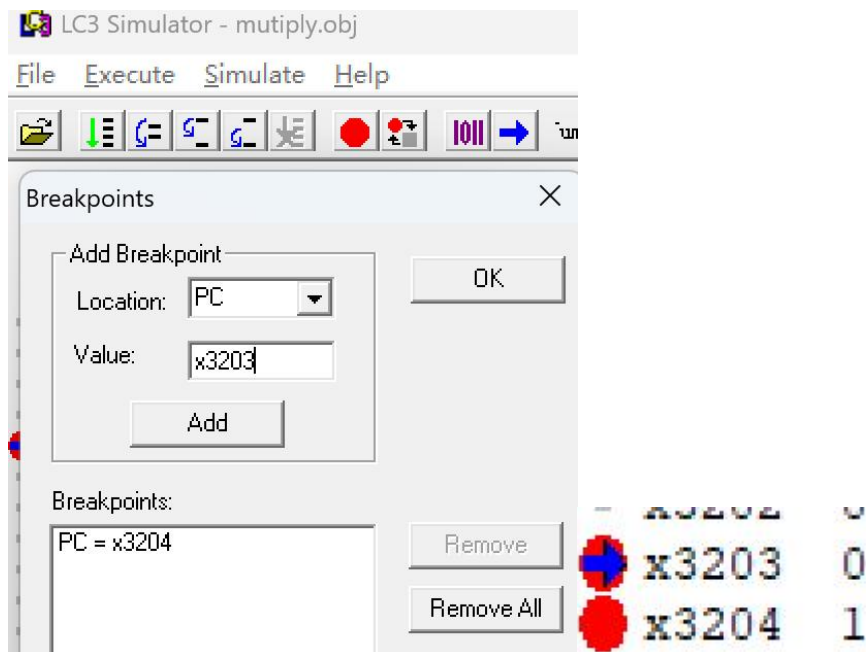
接着 R2 为 5

R0	x0000	0	R4	x0005	5	PC	x3202	12802
R1	x0000	0	R5	x0003	3	IR	x1484	5252
R2	x0005	5	R6	x0000	0	PSR	x8001	-32767
R3	x0000	0	R7	x0000	0	CC	P	
▪ x3200	0101010010100000	x54A0				AND	R2, R2, #0	
▪ x3201	0001010010000100	x1484				ADD	R2, R2, R4	
▶ x3202	0001101101111111	x1B7F				ADD	R5, R5, #-1	
▪ x3203	0000011111111101	x07FD				BRZP	x3201	
▪ x3204	1111000000100101	xF025				TRAP	HALT	

设置断点进行调试：

点击 **OK** 后，会出现两个断点。

观察寄存器变化。此时蓝色箭头和 PC 都指向 x3203，R4 保持不变，R5 变为 x0002，R2 变为 x0005。由于状态码为 P=1，说明结果为正，程序继续执行时分支指令会发生跳转。



再次点击运行并关闭弹出的窗口，继续观察寄存器的值。此时程序已经完成两次循环，R5 变为 x0001，R2 的值为 x000A，即十进制 10。由于状态码仍为 P=1，循环还会继续执行一次。

R0	x0000	0	R4	x0005	5	PC
R1	x0000	0	R5	x0001	1	IR
R2	x000A	10	R6	x0000	0	PSR
R3	x0000	0	R7	x0000	0	CC

x3200	0101010010100000	x54A0	AND
x3201	0001010010000100	x1484	ADD
x3202	0001101101111111	x1B7F	ADD
x3203	0000011111111101	x07FD	BRZP
x3204	1111000000100101	xF025	TRAP

继续运行后，R5 变为 0，R2 变为 x000F，即十进制 15。此时已经完成 $3 \times 5 = 15$ 的计算，程序本应停止循环。但由于状态码为 Z=1，原来的分支指令仍会跳转，导致程序多执行一次循环，因此这里存在错误

R2	x000F	15	R6	x0000	0	PSR	x8002	-32766
R3	x0000	0	R7	x0000	0	CC	Z	

x3200	0101010010100000	x54A0	AND	R2, R2, #0
x3201	0001010010000100	x1484	ADD	R2, R2, R4
x3202	0001101101111111	x1B7F	ADD	R5, R5, #-1
x3203	0000011111111101	x07FD	BRZP	x3201

将 PC 设置为 x3203，然后点击 Add，这样断点列表中就有两条，两条都和 PC 有关，在 PC 为 x3203 时或 x3204 时，模拟器都会暂停，随后把 PC 和 R5 分别设置为 x3200、x0003

R4	x0004	4	PC	x3200	12800
R5	x0003	3	IR	x07FD	2045
R6	x0000	0	PSR	x8004	-32764

因此是分支部分出现错误，我们可以对源码的这一部分进行修改。修改结果为

```
0011001000000000
0101010010100000
0001010010000100
0001101101111111
0000001111111101
1111000000100101
```

R0	x7FFF	32767	R4	x0005	5
R1	xFFFF	-1	R5	x0000	0
R2	x000F	15	R6	x0000	0
R3	x0000	0	R7	xFD75	-651
PC	xFD79	-647	IR	xB02C	-20436
PSR	x8001	-32767	CC	P	

重新在 simulator 中运行，可获得正确答案。

Example2

先将错误程序导入 simulator，如下图

R0	x0000	0	R4	x0000	0	PC	x3000	12288
R1	x0000	0	R5	x0000	0	IR	x0000	0
R2	x0000	0	R6	x0000	0	PSR	x8002	-32766
R3	x0000	0	R7	x0000	0	CC	Z	
→ x3000	1111000000100011	xF023	TRAP	IN				
▪ x3001	0001001000100000	x1220	ADD	R1, R0, #0				
▪ x3002	1111000000100011	xF023	TRAP	IN				
▪ x3003	0001010000000001	x1401	ADD	R2, R0, R1				
▪ x3004	1110000000000100	xE004	LEA	R0, MSG				
▪ x3005	1111000000100010	xF022	TRAP	PUTS				
▪ x3006	0001000010100000	x10A0	ADD	R0, R2, #0				
▪ x3007	1111000000100001	xF021	TRAP	OUT				
▪ x3008	1111000000100101	xF025	TRAP	HALT				
▪ x3009	0000000001010100	x0054	MSG	NOP				
▪ x300A	0000000001101000	x0068	NOP					

在 x3008 设置断点，在 console 窗内输入 4 和 3。


```

■ x3007  1111000000100001  xF021
■ x3008  1111000000100101  xF025
■ x3009  00000000001010100  x0054  MSG
■ X LC3 Console
■ X
■ X
■ X Input a character>
■ X
■ X

```

结果发现程序错误：

```

input a character>4

input a character>3
The sum of those two numbers isg
----- Halting the processor -----

```

接下来开始调试：

R0	x0067	103	R4	x0000	0
R1	x0034	52	R5	x0000	0
R2	x0067	103	R6	x0000	0
R3	x0000	0	R7	x3008	122

x3000	1111
x3001	0001
x3002	1111
x3003	0001
x3004	1110
x3005	1111
x3006	0001
x3007	1111
x3008	1111
x3009	0000
x300A	0000

Breakpoint Encountered


PC = x3008

确定

我们可以看到这个报错中：，R0 中给出的值是 x34.当你输入的是“3”时，显示的是 x33。我们把这些值相加，结果是 x67。查看 ASCII 表，x67 代表的是“g”。

因此我们只需要以下字段，就可以把数据从 ASCII 中提取出来。

ASCII.FILL x30 ;mask: 转换成 ASCII

MEGASCIIFILL xFFD0 ; mask: -x30

重新将改后文件放入 simulator

R0	x7FFF	32767	R4	x0000	0	PC	xFD79	-647
R1	xFFFF	-1	R5	xFFD0	-48	IR	xB02C	-20436
R2	x0037	55	R6	x0030	48	PSR	x8001	-32767
R3	x0000	0	R7	xFD75	-651	CC	P	
→	xFD79	00100000000000011	x2003			LD	R0, xFD7D	
▪	xFD7A	00100010000000011	x2203			LD	R1, xFD7E	
▪	xFD7B	00101110000000011	x2E03			LD	R7, xFD7F	
▪	xFD7C	11000001110000000	xC1C0			RET		

结果正确:

```
Input a character>3

Input a character>4
The sum of those two numbers is 7
----- Halting the processor -----
```

四、实验结论

本次实验通过对 LC-3 仿真器的安装、程序输入、编译运行以及调试过程的实践，全面掌握了 LC-3 开发环境的基本使用方法。在实验过程中，分别采用二进制、十六进制和汇编语言三种方式完成程序的编辑与转换，加深了对机器指令表示形式的理解。同时，通过对示例程序的运行与调试，熟悉了断点设置、单步执行以及寄存器状态观察等调试手段。

在 Example1 中，通过分析循环执行过程和状态码变化，发现分支指令设计不合理导致多执行一次循环，并通过修改指令成功解决问题，提高了对程序逻辑和条件码控制的理解。在 Example2 中，通过分析 ASCII 码与数值之间的差异，定位错误来源并利用偏移量修正输入数据，实现了字符到数值的正确转换。

总体而言，本实验不仅提升了对 LC-3 指令系统和程序执行机制的理解，还增强了程序调试与错误分析能力，为后续深入学习计算机系统奠定了基础。

指导教师批阅意见:

成绩评定：

指导教师签字：
年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。